

Open Science and Reproducible Workflows

OceanHackWeek 2026

What is Open Science

The principles of open science are to make ... scientific research transparent, available, and reproducible.

- NASA Open Science

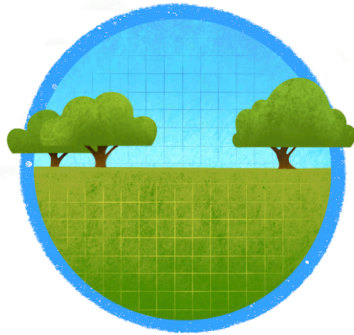
- **Open Data**
- **Open Source tools**
- **Open Code**
- Open Access publications

Reproducibility

Open Data Science

The tooling and people enabling reproducible, transparent, and inclusive practices for data-intensive science

Tooling
tools & practices



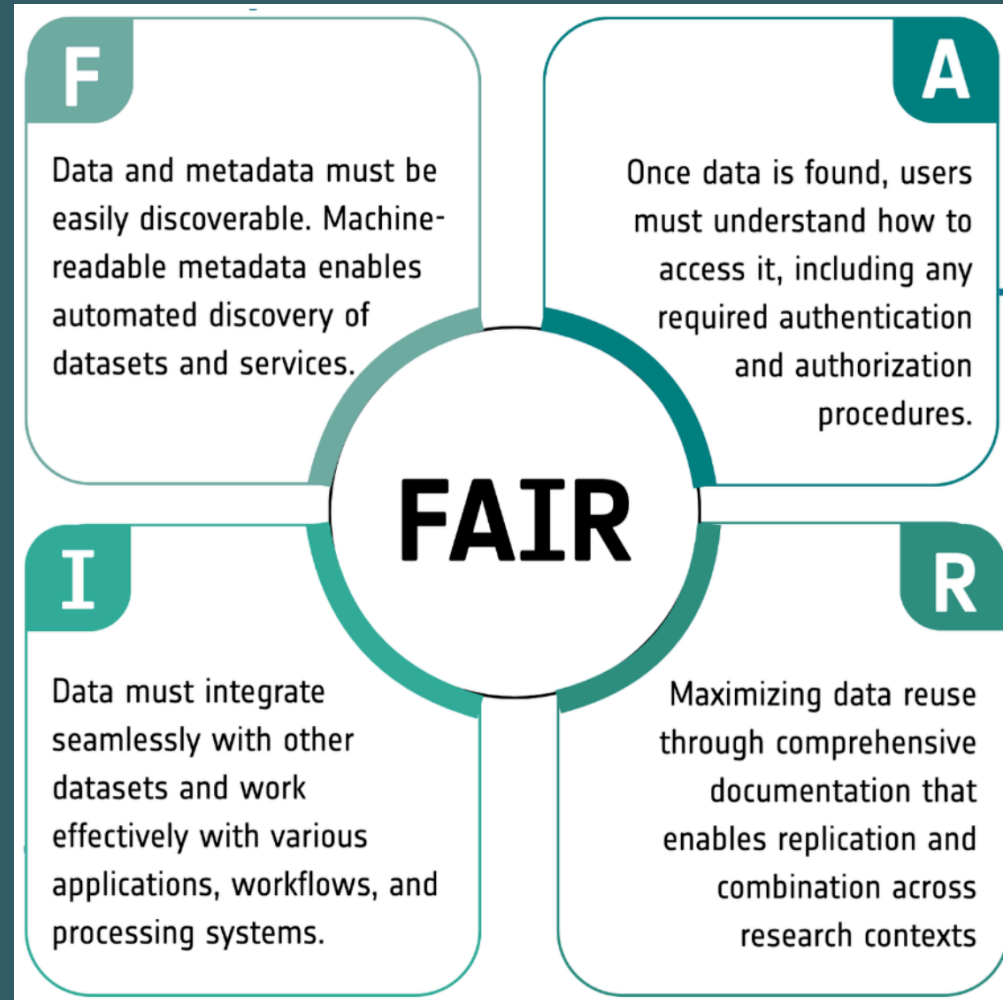
People
teams & community



Open Data

FAIR Principles

- Findable
- Accessible
- Interoperable
- Reusable



CARE Principles



<https://www.gida-global.org/careprinciples>

WHAT ARE **INDIGENOUS DATA**?

Data, information and knowledges, in any format, that impacts Indigenous Peoples, nations, and communities at the collective and individual levels:

DATA ABOUT OUR NON-HUMAN RELATIONS

Land, water,
geology, titles, air,
soil, sacred
ecosystems,
territories, plants,
animals, etc.

USINDIGENOUSDATA.ORG
@USIDSN

DATA ABOUT US AS INDIVIDUALS

Administrative, legal,
health, social,
commercial,
corporate, services,
etc.

Informed by British Columbia First Nations Data Governance Institute - BCFNDGI.COM

DATA ABOUT US AS COLLECTIVES

Traditional and
cultural information,
languages
knowledge systems,
ancestral and clan
knowledges, etc.

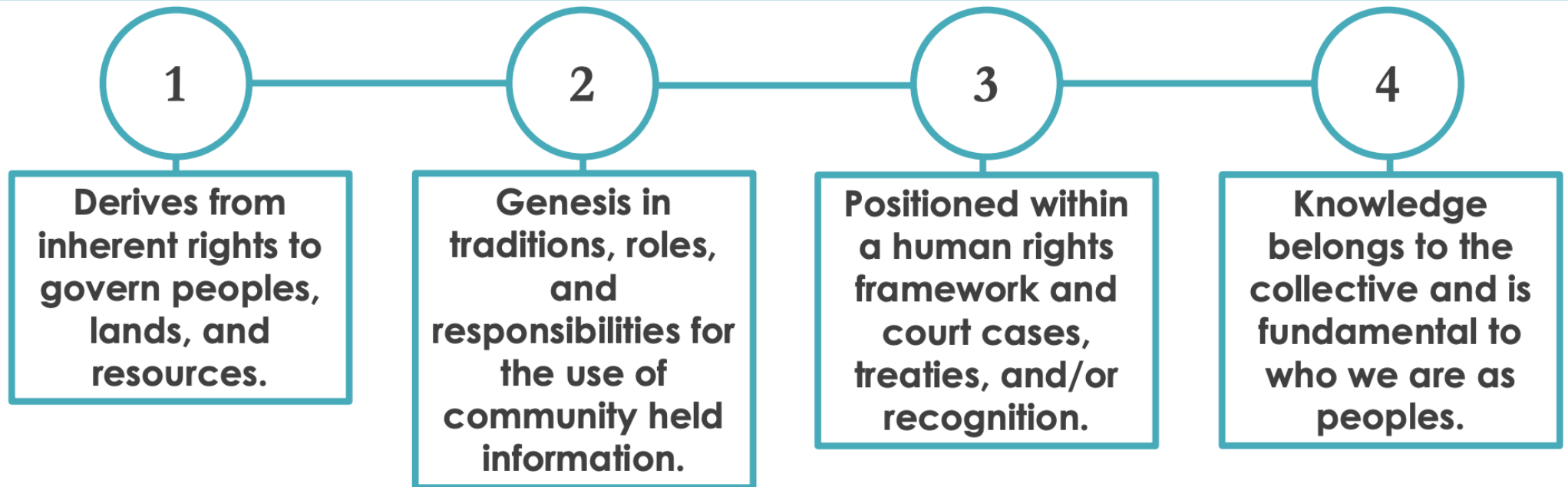
GIDA-GLOBAL.ORG
@GidaGlobal

This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

Reused under CC-BY 4.0 license, from <https://www.gida-global.org/careprinciples>

INDIGENOUS DATA SOVEREIGNTY

The *right* of Indigenous Peoples and nations to govern the collection, ownership, and application of their own data.



See Kukutai T & Taylor J. (Eds). (2016). Indigenous Data Sovereignty. Canberra: Australian National University Press.

USINDIGENOUSDATA.ORG | @USIDSN

GIDA-Global.ORG | @GIDAGlobal

This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

Reused under CC-BY 4.0 license, from <https://www.gida-global.org/careprinciples>

The First Nations Principles of OCAP®

The First Nations principles of ownership, control, access, and possession – more commonly known as OCAP® – assert that First Nations have control over data collection processes, and that they own and control how this information can be used.

<https://fnigc.ca/ocap-training/>

Open Source Tools



Code: Thinking for Reuse

- Often we are (or think we are) only doing our data work on our own
- When we want to facilitate collaboration or focus on reproducibility, we need new strategies:
 - Project organization
 - File naming conventions and file path specification
 - Communication methods

Prerequisite: Save code in scripts

Saving code is an absolute requirement for reproducibility. (Future you, future us)

- Save commands as scripts (`.R/` `.py`) or notebooks (`.qmd/` `.ipynb`).
- It doesn't have to be polished.
- Just save it!
- Everything that really matters should be achieved through code that you save
- Contrast: Series of unrecorded mouse clicks

The process is important, the product is just an outcome

- Outputs should be treated as disposable
- Scripts are the source of truth

Adopt a project-oriented workflow

Why

- You *and your code* can find all parts of your project
- collaborate, communicate, distribute

How

- Single dedicated directory
- RStudio Project or VSCode Workspace
- Git repo, probably syncing to a remote

Organize your work into projects

One folder per project.

- Report? Analysis? Notebook? Website? Whatever.
- Can be the same unit as a GitHub Repo.
- All paths are relative to the project's folder.

What does it mean to work in a Project?

- dedicated **R** or **Python** process running in Project directory
- IDE's file browser pointed at Project directory
- working directory set to Project directory

Often just a project folder that's been opened in its own window via *Open Folder* or similar

Intentionally Choose a Project Structure

```
1 foofy/
2 |— pyproject.toml
3 |— pixi.lock
4 |— README.md
5 |— LICENSE.md
6 |— foofy-paper.qmd
7 |— data/
8 |   |— raw/
9 |   |   |— raw_foofy_data.csv
10 |   |— processed/
11 |   |   |— foofy_clean.csv
12 |— src/
13 |   |— foofy/
14 |   |   |— get-data.py
15 |   |   |— clean.py
16 |   |   |— plotting.py
17 |— notebooks/
18 |   |— explore.ipynb
19 |— figs/
20 |   |— fig.png
```

Name files deliberately

Jenny Bryan's **3 rules for Naming Things**

- machine readable (letters, numbers, “_”, and “-”)
- human readable
- plays well with default ordering

NO

myabstract.docx
 Joe’s Filenames Use Spaces and Punctuation.xlsx
 figure 1.png
 fig 2.png
 JW7d^(2sl@deletethisandyourcareerisoverWx2*.txt

YES

2014-06-08_abstract-for-sla.docx
 joes-filenames-are-getting-better.xlsx
 fig01_scatterplot-talk-length-vs-interest.png
 fig02_histogram-talk-attendance.png
 1986-01-28_raw-data-from-challenger-o-rings.txt

Deliberate use of “_” and “-” allows us to recover meta-data from the filenames.

```
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H01.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H02.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_H03.csv
2013-06-26_BRAFWTNEGASSAY_Plasmid-Cellline-100-1MutantFraction_platefile.csv
```

```
> flist <- list.files(pattern = "Plasmid") %>% head
> stringr::str_split_fixed(flist, "[_\\.]", 5)
      [,1]      [,2]      [,3]      [,4]      [,5]
[1,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "A01" "csv"
[2,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "A02" "csv"
[3,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "A03" "csv"
[4,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "B01" "csv"
[5,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "B02" "csv"
[6,] "2013-06-26" "BRAFWTNEGASSAY" "Plasmid-Cellline-100-1MutantFraction" "B03" "csv"

      date      assay      sample set      well
```

This happens to be R but also possible in the shell, Python, etc.

Practice “safe paths”

The chance of `setwd()` (or `os.chdir()`) having the desired effect – making the file paths work – for anyone besides its author is $\sim 0\%$.

R Python

```
1 setwd("C:\\Users\\andy\\cuddly_broccoli\\verbose\\foofy\\data")
2 df <- read.csv("raw_foofy_data.csv")
3 p <- ggplot(df, aes(x, y)) + geom_point()
4 ggsave("C:\\Users\\andy\\cuddly_broccoli\\ambiguous\\fig.png")
```

- It's also unlikely to work for the author one or two years or computers from now.
- Hard-wired, absolute paths make a project brittle. Such code does not travel well across time or space.

Practice “safe paths”

- Work in a project and use relative paths
- Relative to a **stable base**, not **absolute paths**.
- Use **file system functions** to assemble paths, not `paste()`, `f` strings, etc.

R

```
1 data_path <- file.path("data", "my_data.csv")
2
3 my_data <- read.csv(data_path)
```

Python

```
1 from pathlib import Path
2
3 my_data = pd.read_csv(Path("data") / "my_data.csv")
```

Version Control

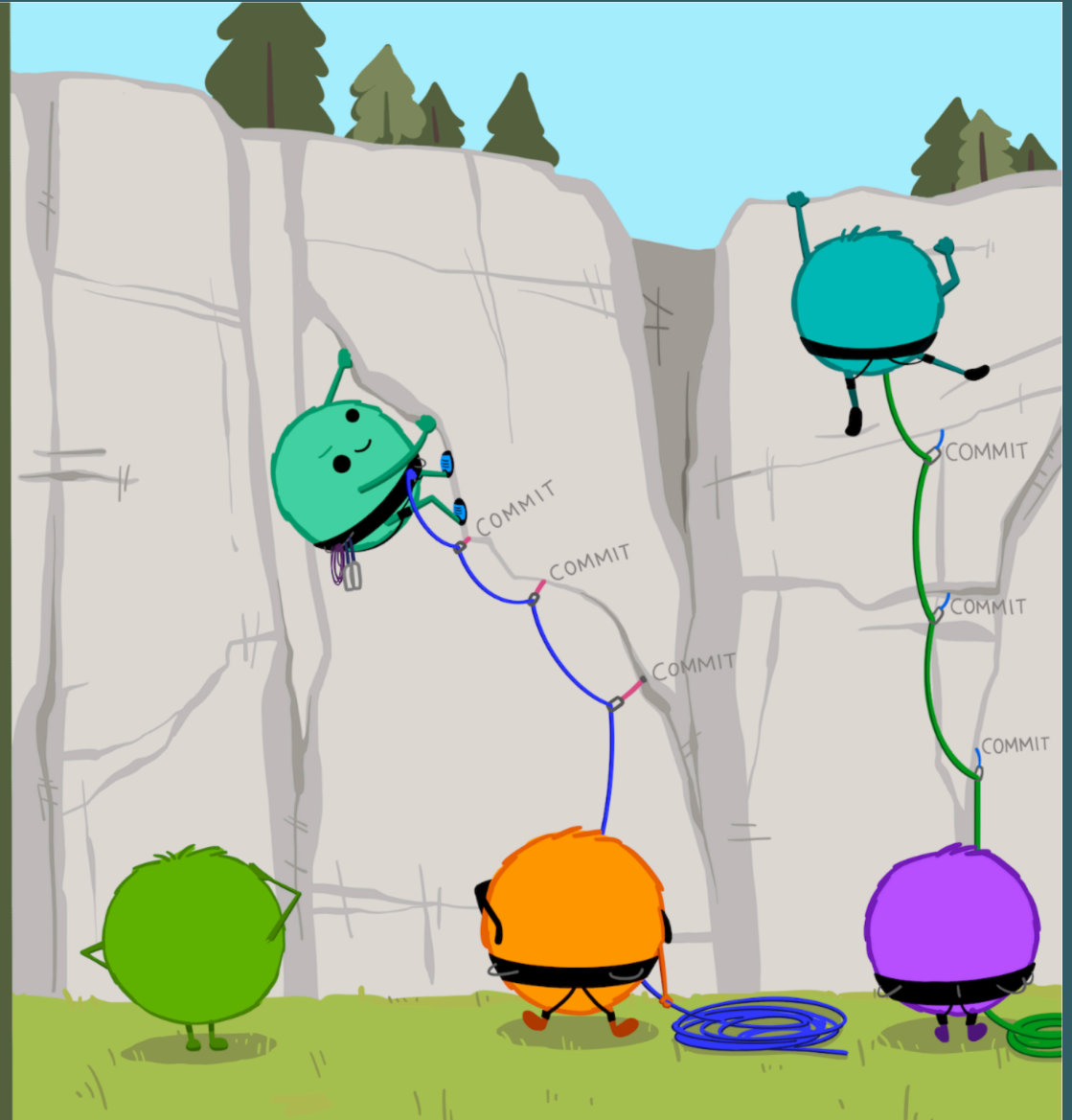


Using a Git commit^{*} is like using anchors and other protection when climbing...**if you make a mistake, you can't fall past the previous commit.**

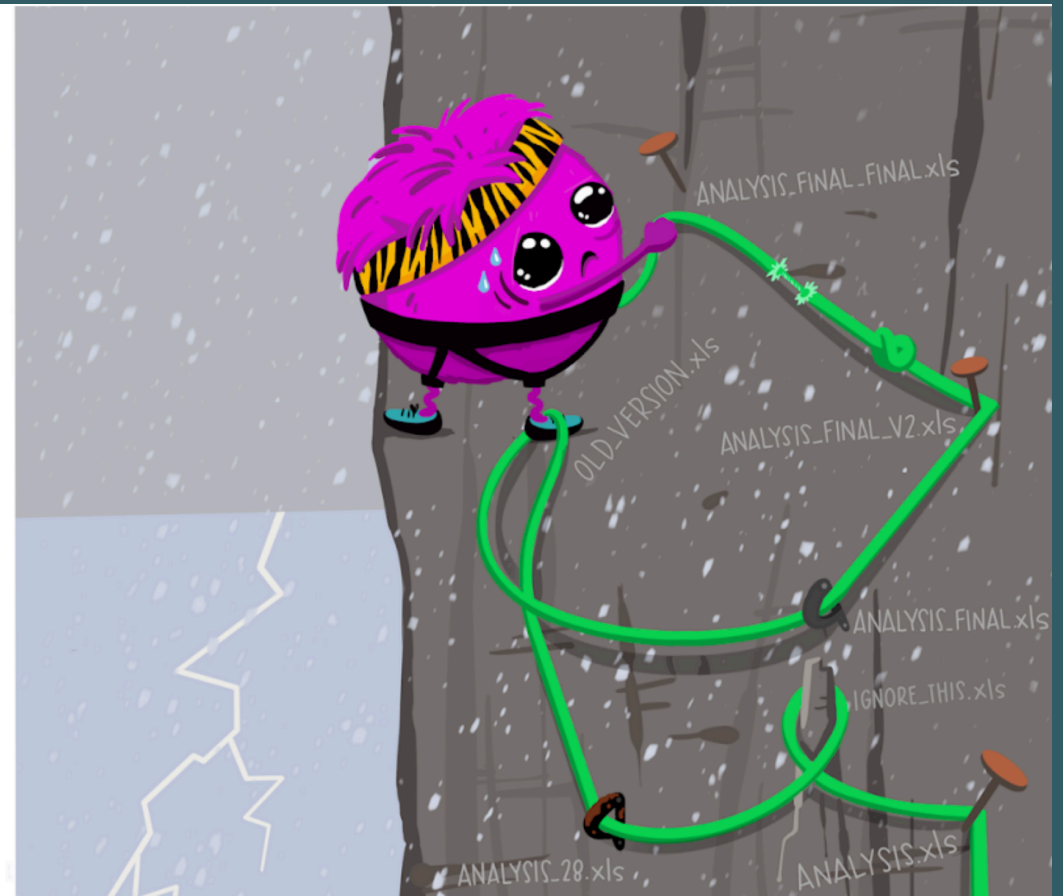
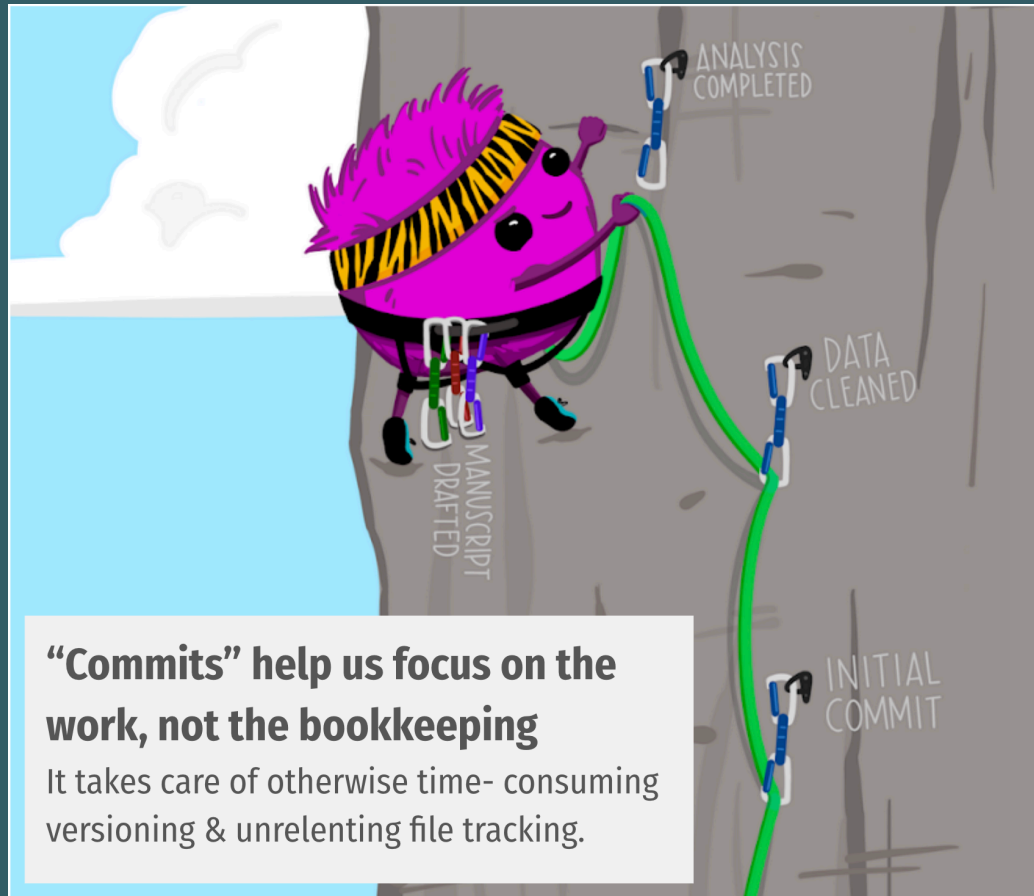
Commits are also helpful to others, because **they show your journey, not just the destination.**”

— HADLEY WICKHAM & JENNY BRYAN

Wickham & Bryan, RPackages (<https://r-packages.org/preface.html>)



Artwork by Allison Horst, CC-BY 4.0



Artwork by [Allison Horst](#), CC-BY 4.0

GitHub: README .md

The first thing a human (including future you) sees when they open the repo.

- What is this project, and why does it exist?
- How do I install/set it up and run it?
- How do I cite it, or who do I contact with questions?

Treat it as living documentation – update it as the project evolves.

GitHub: LICENSE.md

*By default, a GitHub repo without an open source licence applied to it is **not open source** (All Rights Reserved).*

- MIT is a good, permissive license for most projects
- Apache 2.0 is similar, but adds an explicit patent grant

Work Projects

1 Copyright (c) 2026 His Majesty the King in Right of Canada, as represented

Personal projects

1 Copyright (c) 2026 Andy Teucher

Other important files

CONTRIBUTING.md

CODE_OF_CONDUCT.md

Dependency Management

Why manage dependencies?

- Code depends on specific packages – **often specific versions** of them
- “Works on my machine” isn’t reproducible
- A **lockfile** records the exact environment (packages + versions) so it can be recreated on another machine, months or years later
- Goal: record/restore the *exact* environment a project needs

renv (R) and pixi (Python)

Both create a **project-local** environment and a lockfile you commit to the repo.

R (renv)

Python (pixi)

```
1 renv::init()      # start using renv in this project
2 renv::snapshot()  # record package versions -> renv.lock
3 renv::restore()   # reinstall exact versions from renv.lock
```

Produces **renv.lock** + a project-local **renv/library**.

Reproducible Reporting

Exact results can be reproduced if given access to the original data, software, and code.

If we can put our code and narrative text in a single document, it means:

- You don't have to copy and paste figures and tables into reports
- Reports are automatically updated when data or analyses change
- Human error is reduced
- Enhanced transparency and reproducibility

Iterating & communicating with "literate programming"

Quarto, Jupyter notebooks combines code + text + output (tables, figures)

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
```{r}  
summary(cars)
```
```

Including Plots

You can also embed plots, for example:

```
```{r pressure}  
#| echo: false
plot(pressure)
```
```

Note that the `echo: FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Simple text formatting

Code – R, Python, SQL,
bash, others

Analyses & figures in the same
place as reporting document:
saves time as you iterate!

Enables good practices for
reproducibility & versioning

Learn: [Quarto.org](https://quarto.org); jupyter.org

```

---
title: "Untitled"
author: "Andy Teucher"
date: "2024-11-18"
---

## Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

## Running Code

When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```{r}
summary(cars)
```

## Including Plots

You can also embed plots, for example:

```{r pressure}
#| echo: false
plot(pressure)
```

Note that the `echo: FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```

Our Quarto file renders to:

Word!

PDF!

Untitled

Andy Teucher
2024-11-18

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

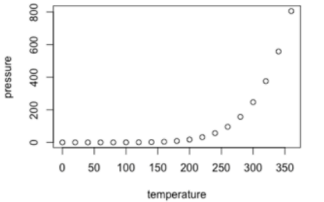
When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
summary(cars)
```

| speed | dist |
|--------------|----------------|
| Min. : 4.0 | Min. : 2.00 |
| 1st Qu.:12.0 | 1st Qu.: 26.00 |
| Median :15.0 | Median : 36.00 |
| Mean :15.4 | Mean : 42.98 |
| 3rd Qu.:19.0 | 3rd Qu.: 56.00 |
| Max. :25.0 | Max. :120.00 |

Including Plots

You can also embed plots, for example:



Note that the echo: FALSE parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Untitled

Andy Teucher
2024-11-18

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

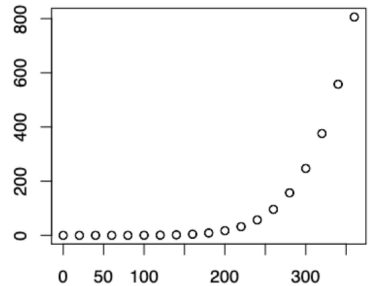
When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
summary(cars)
```

| speed | dist |
|--------------|----------------|
| Min. : 4.0 | Min. : 2.00 |
| 1st Qu.:12.0 | 1st Qu.: 26.00 |
| Median :15.0 | Median : 36.00 |
| Mean :15.4 | Mean : 42.98 |
| 3rd Qu.:19.0 | 3rd Qu.: 56.00 |
| Max. :25.0 | Max. :120.00 |

Including Plots

You can also embed plots, for example:



Note that the echo: FALSE parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Imagine never copy-pasting a graph into your report again!!!!

Quarto can also manage citations, cross- referencing figures and section headers.

Quarto

Quarto creates HTML files that can be shared openly on the web

```
---
title: "Untitled"
author: "Andy Teucher"
date: "2024-11-18"
---

## Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see https://quarto.org.

## Running Code

When you click the Render button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```{r}
summary(cars)
```

## Including Plots

You can also embed plots, for example:

```{r pressure}
#| echo: false
plot(pressure)
```

Note that the `echo: FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.
```

Our Quarto file renders to:

HTML!

Untitled

AUTHOR
Andy Teucher

PUBLISHED
November 18, 2024

Quarto

Quarto enables you to weave together content and executable code into a finished document. To learn more about Quarto see <https://quarto.org>.

Running Code

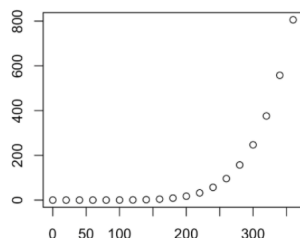
When you click the **Render** button a document will be generated that includes both content and the output of embedded code. You can embed code like this:

```
summary(cars)
```

| speed | dist |
|---------------|----------------|
| Min. : 4.0 | Min. : 2.00 |
| 1st Qu.: 12.0 | 1st Qu.: 26.00 |
| Median : 15.0 | Median : 36.00 |
| Mean : 15.4 | Mean : 42.98 |
| 3rd Qu.: 19.0 | 3rd Qu.: 56.00 |
| Max. : 25.0 | Max. : 120.00 |

Including Plots

You can also embed plots, for example:



Note that the `echo: FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

We can store and distribute html files on **GitHub**, combined as websites, slides, books

>> Reimagine sharing, sci comm, engagement, inclusion

Faylab Lab Manual

1 Welcome!

2 Introduction

3 Lab culture and philosophy

4 Code of Conduct

5 Onboarding

6 Expectations

7 Offboarding

8 Funding

9 Communication

10 Academics

11 Courses

12 Computing resources

13 UMass Dartmouth, SMAST, and Fay Lab Resources

Faylab Lab Manual

This is the lab manual for the Fay Lab at the University of Massachusetts Dartmouth School for Marine Science and Technology

AUTHOR
Gavin Fay & Ashleigh Novak

PUBLISHED
July 12, 2022

1 Welcome!

Welcome to our book of magical secrets.

This is the lab manual for the Fay Lab at the University of Massachusetts Dartmouth School for Marine Science and Technology.

The focus of our work centers around developing interdisciplinary modeling approaches to extend the scope of applications for fisheries and ecosystem-based management. More about our group's research activity can be found on our [website](#).

This lab manual resource is intended to provide an overview for lab members and others about how we do our work, and our expectations for our team. It is also a space to document institutional knowledge and

Suddenly you can share a URL rather than attaching a file!

And that same URL will update rather than re-attaching a new version of the file!

Openscapes blogs: [An RMarkdown Chat with Alison Hill](#); [Data Science as an Entryway to Open Publishing](#); [Quarto Keynote at the RStudio Conference](#)

Reproducible Workflows

-  Open Data (FAIR + CARE)
-  Open Source Tools
-  Project-oriented Workflows
-  Open Code + Version Control
-  Dependency management (e.g., `uv/pixi/conda/renv`)
-  Literate Programming (Quarto, Jupyter Notebooks)
- Containerization (e.g., Docker)

Checklists: https://github.com/oceanhackweek/ohwyy_proj_template/issues

Discussion

- What is one step you can take right now to make your work more reproducible?
- What is the biggest challenge in your work to making it more reproducible?

Slides: [pdf](#) / [Quarto source](#)